ソフトウェア工学特論 講義資料

第9回 チーム開発①：要件定義

- 機能要件と非機能要件の分類
- Issue駆動のタスク落とし込み
- KPT自己評価の開始（毎週提出）

目次

- 要件定義の基礎
- IssueへのToDo落とし込み
- KPT自己評価の運用

要件定義の基礎

今回の目的と到達目標

今回の目的

- チームで要件を整理・言語化する
- IssueでToDoを洗い出す
- KPT自己評価を開始する

到達目標

- [R3-未到達] メンバーとして担当作業を遂行
- [R3-標準] 要件定義の工程を管理

機能要件と非機能要件

区分	内容	例
機能要件	システムが **何をする** か	「ログインできる」「予約一覧を表示」
非機能要件	**どう振る舞う** か	応答時間・利用環境・セキュリティ

- 両方を書かないと後工程で齟齬が出る
- 非機能は **数値化** する（例：3秒以内、同時10人）
- まず機能要件を列挙、その後に制約で非機能を整理する

要件定義書のひな型

```
## システム概要
〇〇を管理するWebアプリ

## 機能要件
- F-01 ユーザー登録
- F-02 ログイン
- F-03 予約作成
- F-04 予約一覧表示

## 非機能要件
- 応答時間：1操作あたり2秒以内
- 対象ブラウザ：Chrome 最新版
- 同時接続数：5ユーザー
```

IDを振る と後でIssue・設計書から参照しやすい。

IssueへのToDo落とし込み

要件 → Issue の粒度

- 要件ID1つに対して **Issueを1〜数件** 作成
- 1 Issue = 1 PRでマージできる単位が目安
- 大きすぎる場合は **サブタスクに分解**
- 小さすぎる場合は関連要件をまとめる
- ラベル例：`feature` / `bug` / `docs` / `design`

Issueテンプレート例

```
## 対応要件
F-03 予約作成

## 完了条件
- [ ] 予約登録フォーム画面
- [ ] 入力バリデーション
- [ ] DB保存処理
- [ ] 成功メッセージ表示

## 担当
@user1
```

チェックリスト式の **完了条件** がPRレビュー時の判定基準になる。

KPT自己評価の運用

KPTとは

項目	意味
Keep	続けたいこと・うまくいったこと
Problem	問題・困ったこと
Try	次に試したい改善

- 第9〜14週、**毎週末に1回** 提出
- 評価：1点×回数（提出漏れ・空白・意味のない文章は0点）
- 第15週のレポート③の材料になる

KPT提出フォーマット

```
## 第9週 KPT
### Keep
- Issueを要件に紐付けて作れた

### Problem
- 要件と非機能の切り分けで迷った

### Try
- 次週は非機能要件を数値化してから書く
```

具体的な事象 を書くことが第15週のレポート③で効いてくる。

今回のまとめ

- 機能要件と非機能要件を分けて言語化（非機能は数値化）
- 要件にIDを振り、Issueに1〜数件で対応させる
- KPTは毎週提出、意味のある文章を書く
- Issueの完了条件がレビュー基準になる

今回カバーしたMCC項目

- V-D-4 コンピュータシステム

次回予告

- 第10回: チーム開発② 基本設計（画面・DB・システム構成）